

Enabling the Optional Jactett Sidecar

Field	Value
Document	docs/guides/jackett-sidecar.md
Revision	1
Last updated	2026-06-08
Status	Current
Scope	Operator guide to running the optional Jackett (+ FlareSolvrr) sidecar behind lava-api-go

Grounded in `lava-api-go/internal/config/config.go`, `lava-api-go/internal/router/router.go`, `lava-api-go/internal/handlers/v1/jackett.go`, `tools/lava-containers/docker-compose.jackett.yml`, and `docs/requests/improvements/lava_jackett_program_worklog.md`. Not-yet-wired items are marked PENDING (§11.4.6 — no guessing).

What the Jakkett sidecar is

Jackett is a Torznab indexer proxy: it turns dozens of tracker sites into a single normalized search API. Lava can run Jackett as an **optional sidecar** so that lava-api-go can search indexers Lava has no native scraper for.

Key topology rule (from `docker-compose.jackett.yml`): the Android app **never talks to Jackett directly**. Jackett (port 9117) and FlareSolvrr (port 8191) stay on the internal `lava-net` bridge and are **not** published to the LAN. The app sees exactly one endpoint — `lava-api-go (_lava-api._tcp, :8443)` — and `lava-api-go` proxies Torznab server-side, holding the Jackett `api_key` so the secret never reaches the device (§6.H).

```
Android app —mDNS(_lava-api._tcp)→ lava-api-go (:8443, host-  
net, TLS+HMAC)  
| (server-side HTTP,  
holds api key)
```

▼
jackett (:9117) — lava-

net, NOT advertised

|
(only when profile=cloudflare)
▼
flaresolverr (:8191) — lava-

net

The configuration knobs

lava-api-go reads four env vars at boot (internal/config/config.go). The route is **OFF by default** (§6.R — nothing hardcoded, sidecar is opt-in):

Env var	Config field	Default	Meaning
LAVA_API_JACKETT_ENABLED	JackettEnabled	false	Turns the /jackett/search route on.
LAVA_API_JACKETT_URL	JackettBaseURL	(empty)	Sidecar base, e.g. http://lava-jackett:9117.
LAVA_API_JACKETT_APIKEY	JackettAPIKey	(empty)	Jackett api_key — server-side secret only (§6.H).
LAVA_API_JACKETT_DEFAULT_INDEXER	JackettDefaultIndexer	all	Indexer id used when a request omits indexer (all = every configured indexer).

Validation (config.go): when LAVA_API_JACKETT_ENABLED=true, both LAVA_API_JACKETT_URL **and** LAVA_API_JACKETT_APIKEY must be set, or boot fails with: config: LAVA_API_JACKETT_URL and LAVA_API_JACKETT_APIKEY are required when LAVA_API_JACKETT_ENABLED=true.

PENDING: these four LAVA_API_JACKETT_* vars are **not yet present in .env.example** at the time of writing — only the compose-glue vars below are documented there. Set them in your gitignored .env directly.

The single route the app uses

From `internal/router/router.go`, the route is registered **only** when the sidecar is enabled and a client can be built:

```
if deps.Cfg != nil && deps.Cfg.JackettEnabled {
    if jc, err := jackett.NewClient(...); ... {
        jh := vlhandlers.NewJackettHandler(jc,
            deps.Cfg.JackettDefaultIndexer)
        engine.GET("/jackett/search", jh.GetSearch)
    }
}
```

GET `/jackett/search` (handler `jackett.go`):

- **Query params:** `q` (required), `indexer` (optional — defaults to the configured default indexer / `all`).
- **Responses:** 200 JSON (mapped into the same `provider.SearchResult` DTO every other Lava search surface returns, stamped with provider id `jackett`), 400 if `q` is missing, 502 on a sidecar error.

The app therefore consumes Jackett results exactly like any native provider's results — it has no awareness of Jackett at all.

Bringing the sidecar up

Orchestration is owned by `tools/lava-containers/cmd/lava-containers`. Do **not** bring `docker-compose.jackett.yml` up on its own — merge it with the root compose (per the file's own header):

```
docker compose -f ../../docker-compose.yml -f docker-
compose.jackett.yml \
--profile api-go --profile jackett config      # verify the
merged config
```

Two compose profiles (from `docker-compose.jackett.yml`):

- **jackett** — brings up `lava-jackett` only.
- **cloudflare** — additionally brings up `lava-flaresolverr`.

The lava-jackett service

- Image: `${LAVA_JACKETT_IMAGE}` (digest-pin in `.env`; LinuxServer multi-arch).
- Exposed only on the bridge (expose: 9117) — **no ports: mapping to the LAN.**
- Env: `PUID/PGID/TZ` (`LAVA_JACKETT_*`), `AUTO_UPDATE: false` (self-update would drift the digest pin).

- Healthcheck is a real **Torznab t=caps probe** (HTTP 200 + parseable body), not a bare TCP check — so it also validates the `api_key`, not just liveness (§6.B). `start_period` is 40s for the .NET cold load.

Where the `api_key` lives

The Jackett `api_key` is **generated on first run** into the gitignored `/config` volume's `ServerConfig.json` (`{LAVA_JACKETT_CONFIG_DIR:-./jackett-config}:/config`). The Containers glue reads it from there and injects it into `lava-api-go` as `LAVA_API_JACKETT_APIKEY`. It is **never committed, never logged, never sent to the device** (§6.H). The `/config` volume is a secrets store and its host path **MUST** be gitignored.

Cloudflare-protected trackers (FlareSolverr)

Some indexers (notably **IPTorrents**) sit behind a Cloudflare / DDoS-Guard challenge. **FlareSolverr** solves these by running a headless Chromium per concurrent challenge. Because that is the heaviest component in the stack, it is profiled **off** by default — only bring it up (`--profile cloudflare`) when you have an indexer that needs it.

The `lava-flaresolverr` service

- Image: `${LAVA_FLARESOLVERR_IMAGE}`.
- Exposed only on the bridge (`expose: 8191`) — never published to the LAN.
- Env: `LOG_LEVEL`, `HOST=0.0.0.0`, `PORT=8191`, `HEADLESS=true`.
- Healthcheck POSTs `{"cmd":"sessions.list"}` to `/v1` (FlareSolverr has no `/health`; a bare TCP check would be a bluff because the Python proxy binds before Chromium is ready). `start_period` is 60s for Chromium warm-up.
- **Wiring:** Jackett is pointed at FlareSolverr (`http://lava-flaresolverr:8191`) at **indexer-provisioning time** inside Jackett's own config — not via a Lava env var.

Keep the `cloudflare` profile **off** for RuTracker / RuTor / NNM-Club-only sessions — those do not need FlareSolverr, and the Chromium process is RAM-heavy.

Compose-glue env vars (in .env.example)

These configure the containers (distinct from the LAVA_API_JACKETT_* vars that configure lava-api-go itself). All are §6.R-injected — no literals in the compose file:

Env var	Purpose
LAVA_JACKETT_IMAGE	Digest-pinned Jackett image.
LAVA_FLARESOLVERR_IMAGE	Digest-pinned FlareSolverr image.
LAVA_JACKETT_PUID / LAVA_JACKETT_PGID / LAVA_JACKETT_TZ	Container user / timezone.
LAVA_JACKETT_CONFIG_DIR	Host path for the /config secrets volume (gitignored).
LAVA_JACKETT_API_KEY	Read from /config, used by the healthcheck probe and injected into lava-api-go.
LAVA_JACKETT_HEALTH_INDEXER	Indexer id used by the caps healthcheck (default all).
LAVA_FLARESOLVERR_LOG_LEVEL	FlareSolverr log verbosity.

Step-by-step: enable Jackett

1. Set the four LAVA_API_JACKETT_* vars in your gitignored .env (LAVA_API_JACKETT_ENABLED=true, LAVA_API_JACKETT_URL=http://lava-jackett:9117, LAVA_API_JACKETT_APIKEY=<from /config>, optionally LAVA_API_JACKETT_DEFAULT_INDEXER).
2. Set the compose-glue vars (LAVA_JACKETT_IMAGE, LAVA_JACKETT_CONFIG_DIR, LAVA_JACKETT_API_KEY, etc.) in .env.
3. Bring up the jackett profile (merged with the root compose, via the lava-containers CLI). On first run, open the Jackett WebUI **from the host** (it is bridge-internal), add indexers, and copy the generated api_key from ServerConfig.json into LAVA_API_JACKETT_APIKEY / LAVA_JACKETT_API_KEY.
4. For a Cloudflare-protected indexer (e.g. IPTorrents), also bring up the cloudflare profile and point that indexer at http://lava-flaresolverr:8191 inside Jackett.
5. Restart lava-api-go so it picks up the new env and registers /jackett/search. The app's searches now include Jackett results.